

Refer the following link to understand the Parsing and Lexical analysis
<https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/>

Task 1:

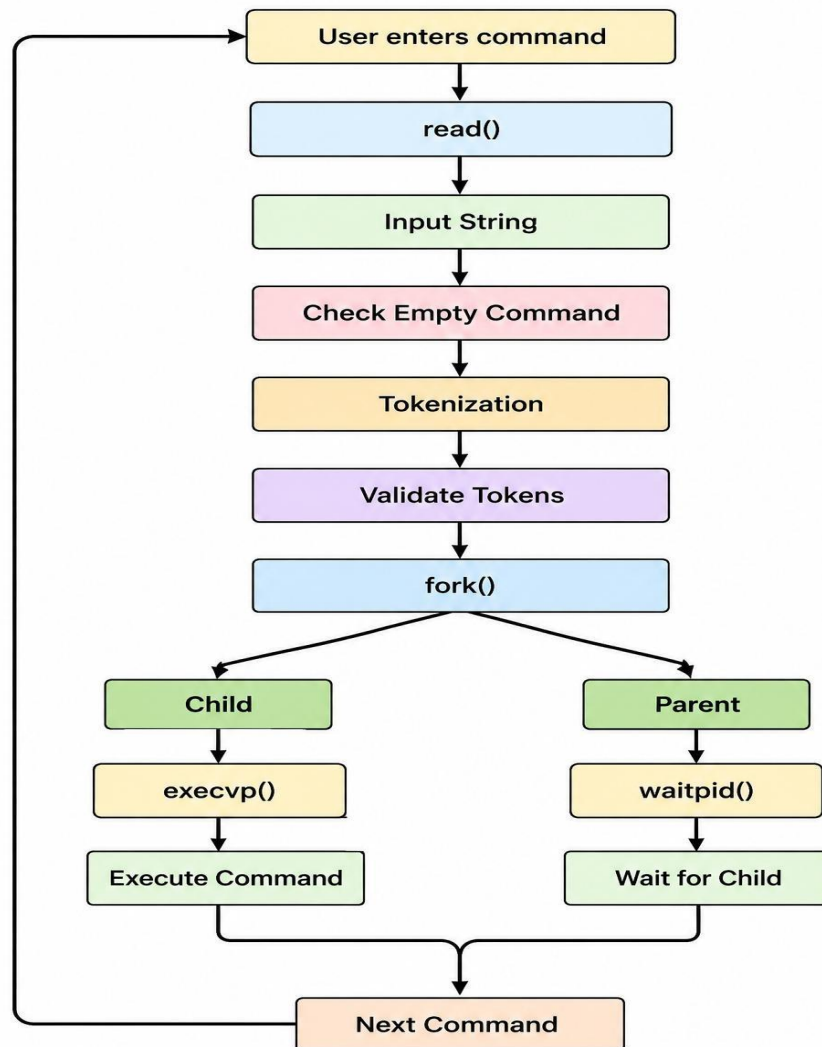
To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

Aim

To split a shell command into tokens and identify the individual arguments.

Program

Flow Chart:



Program

```
#include <stdio.h>
#include <string.h>

#define MAX_TOKENS 20

int main() {
    char input[200];
    char *token;
    char *tokens[MAX_TOKENS];
    int count = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    token = strtok(input, " \\t\\n");

    while (token != NULL && count < MAX_TOKENS) {
        tokens[count++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    if (count == 0) {
        printf("Empty command.\\n");
        return 0;
    }

    printf("\\n--- Parsed Arguments ---\\n");

    for (int i = 0; i < count; i++) {
        printf("Argument %d: %s\\n", i + 1, tokens[i]);
    }

    printf("Tokenization completed successfully.\\n");

    return 0;
}
```

Output

```
root@keerthana:~# cd week4
root@keerthana:~/week4# pwd
/root/week4
root@keerthana:~/week4# nano task1.c
root@keerthana:~/week4# gcc task1.c -o task1
root@keerthana:~/week4# ./task1
myShell> ls -l /home

--- Parsed Arguments ---
Argument 1: ls
Argument 2: -l
Argument 3: /home
Tokenization completed successfully.
root@keerthana:~/week4#
```

Observation

The input command was successfully divided into separate tokens based on whitespace.

Result

The command was successfully tokenized and the individual arguments were displayed.

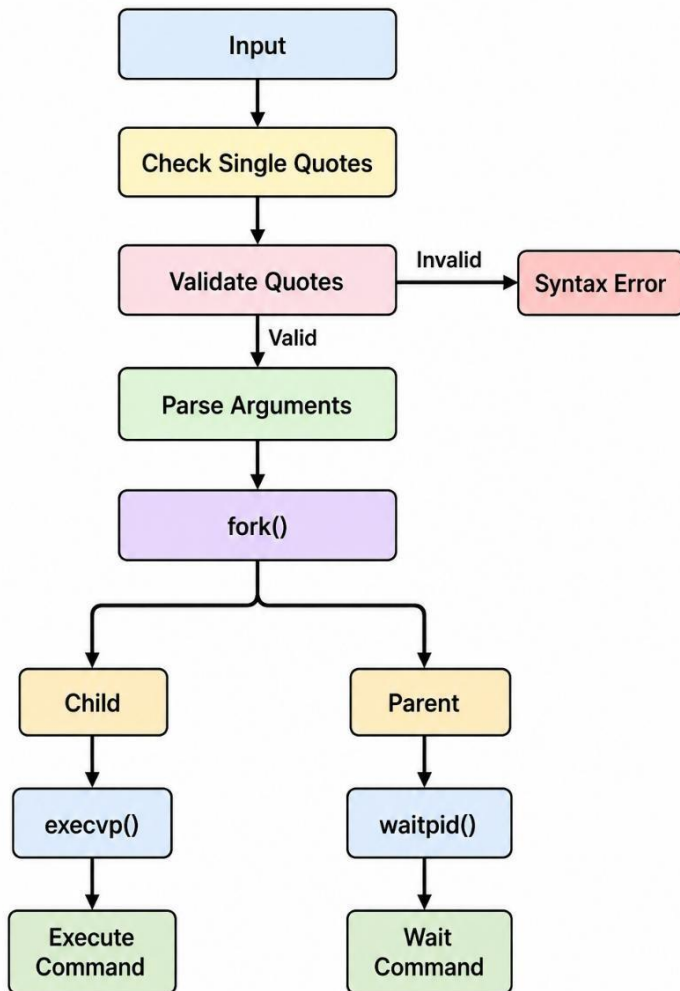
Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

Aim

To handle single quotes in shell commands and detect unmatched quotes.

Flow Chart:



Example Output:

Case1:

myShell> echo Hello

Parsing Result:

Argument 1: echo

Argument 2: Hello

Child PID: 3210

Hello

Command execution completed.

Case2:

myShell> echo 'Hello World

Syntax Error: Unmatched single quote.

Program

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    int quotes = 0;

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '"')
            quotes++;
    }

    if (quotes % 2 != 0) {
        printf("Syntax Error: Unmatched single quote.\n");
        return 1;
    }

    printf("Parsing successful.\n");
    printf("Literal content preserved.\n");

    return 0;
}
```

Output

```
root@keerthana:~/week4# nano task2.c
root@keerthana:~/week4# gcc task2.c -o task2
root@keerthana:~/week4# ./task2
myShell> myShell> echo 'Hello World'^[[D^[[D^[[D^[[D^[[D^[[D^[[D
Syntax Error: Unmatched single quote.
root@keerthana:~/week4# echo 'Hello World'
Hello World
root@keerthana:~/week4# gcc task2.c -o task2
root@keerthana:~/week4# ./task2
myShell> myShell> echo 'Hello World'
Parsing successful.
Literal content preserved.
root@keerthana:~/week4# ./task2
myShell> echo 'Hello World
Syntax Error: Unmatched single quote.
root@keerthana:~/week4# |
```

Observation

Single quotes were recognized and the quoted content was preserved. An unmatched single quote was detected as a syntax error.

Result

Single-quoted strings were successfully handled and invalid quoted input was detected.

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

Aim

To handle double quotes in shell commands, preserve spaces and detect unmatched double quotes.

Example Output:

Case1:

```
myShell> echo "Hello $USER"
```

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Hello raghu

Child Process

PID = 3022

Hello raghu

Child process completed.

Case2:

```
myShell> echo "Hello World
```

Syntax Error: Unmatched double quote.

Program

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char input[200];
```

```
    printf("myShell> ");
```

```
    fgets(input, sizeof(input), stdin);
```

```

int quotes = 0;

for (int i = 0; input[i] != '\0'; i++) {
    if (input[i] == '"')
        quotes++;
}

if (quotes % 2 != 0) {
    printf("Syntax Error: Unmatched double quote.\n");
    return 1;
}

printf("--- Parsed Arguments ---\n");

char *start = input;
int arg = 1;

while (*start != '\0' && *start != '\n') {
    while (*start == ' ')
        start++;

    if (*start == '\0' || *start == '\n')
        break;

    if (*start == '"') {
        start++;
        char *end = strchr(start, '"');

        if (end) {
            *end = '\0';
            printf("Argument %d: %s\n", arg++, start);
            start = end + 1;
        }
    } else {
        char *end = start;

        while (*end != ' ' && *end != '\n' && *end != '\0')
            end++;

        char temp = *end;

```

```

    *end = '\0';

    printf("Argument %d: %s\n", arg++, start);

    if (temp == '\0' || temp == '\n')
        break;

    start = end + 1;
}
}

return 0;
}

```

Output

```

root@keerthana:~/week4# nano task3.c
root@keerthana:~/week4# gcc task3.c -o task3
root@keerthana:~/week4# ./task3
myShell> echo "Hello World"
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
root@keerthana:~/week4# ./task3
myShell> echo "Hello World
Syntax Error: Unmatched double quote.
root@keerthana:~/week4# |

```

Observation

Double-quoted text was treated as a single argument and spaces inside the quotes were preserved. An unmatched double quote was also detected.

Result

Double-quoted strings were successfully parsed and syntax errors were detected.

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

Program

```
#include <stdio.h>
#include <string.h>
int main() {
    char input[200];

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    printf("--- Parsed Input ---\n");

    for (int i = 0; input[i] != '\0' && input[i] != '\n'; i++) {
        if (input[i] == '\\' && input[i + 1] != '\0') {
            i++;
            printf("%c", input[i]);
        } else {
            printf("%c", input[i]);
        }
    }

    printf("\nParsing completed successfully.\n");

    return 0;
}
```

Output

```
root@keerthana:~/week4# nano task4.c
root@keerthana:~/week4# gcc task4.c -o task4
root@keerthana:~/week4# ./task4
myShell> Hello\ World
--- Parsed Input ---
Hello World
Parsing completed successfully.
root@keerthana:~/week4# ./task4
myShell> echo Hello\@World
--- Parsed Input ---
echo Hello@World
Parsing completed successfully.
root@keerthana:~/week4# |
```

Observation

The escape character was recognized and the escaped space and special character were preserved as part of the input.

Result

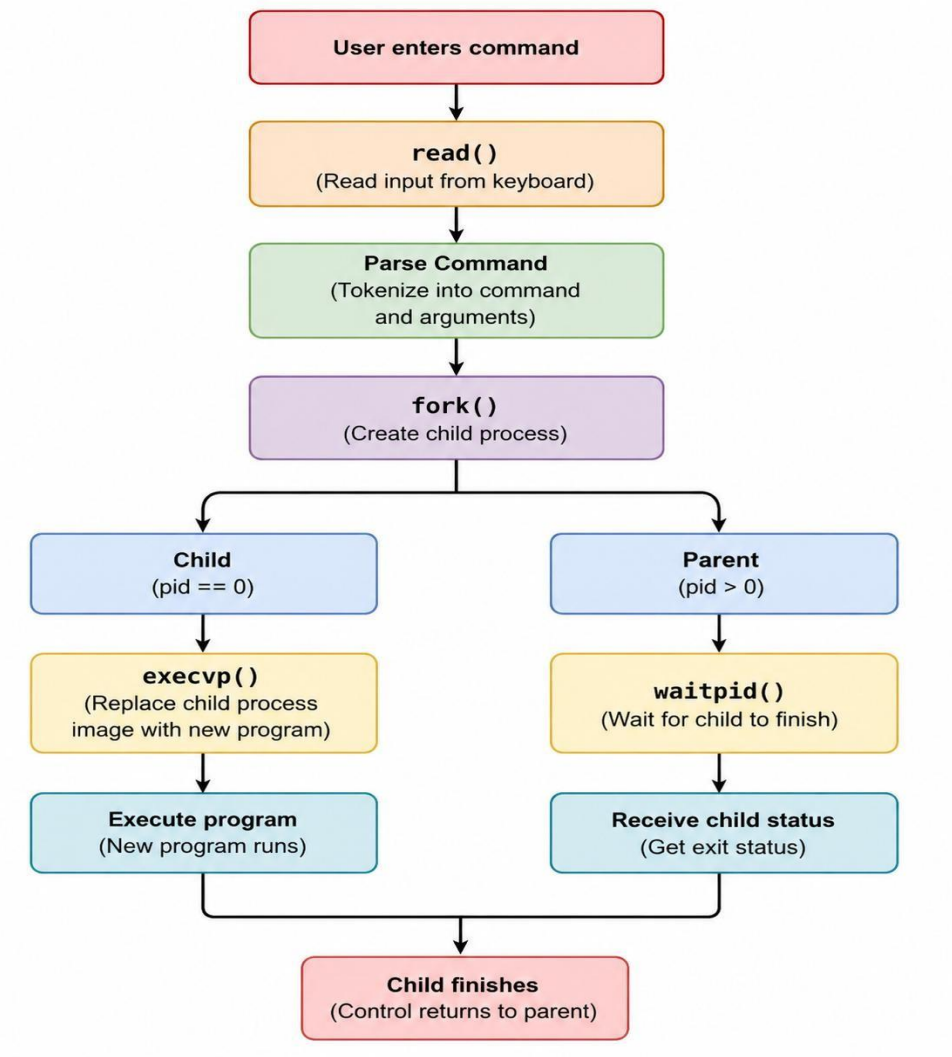
Escape sequences were successfully handled and the original characters were preserved.

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

Aim

To create a child process and execute a command using `fork()` and `exec()`.

Flow Chart:



Sample Output:

Case 1:

myshell\$ ls

Parent Process PID: 1200

Created Child PID: 1201

Child Process PID: 1201

file1.txt

file2.txt

Child exited with status: 0

Case 2:

myshell\$ ls @l

Parent Process PID: 82

Child Process PID: 83

Created Child PID: 83

ls: cannot access '@l': No such file or directory

Child exited with status: 2

Program

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main(int argc, char *argv[]) {
```

```
    if (argc < 2) {
```

```
        printf("Usage: %s <command> [arguments...]\n", argv[0]);
```

```
        return 1;
```

```
    }
```

```
    pid_t pid = fork();
```

```
    if (pid < 0) {
```

```
        perror("fork failed");
```

```
        return 1;
```

```
    }
```

```
    if (pid == 0) {
```

```
        printf("Child Process PID: %d\n", getpid());
```

```
        execvp(argv[1], &argv[1]);
```

```
        perror("exec failed");
```

```
        exit(1);
```

```

    }
    else {
        int status;

        printf("Parent Process PID: %d\n", getpid());
        printf("Created Child PID: %d\n", pid);
        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
            printf("Child exited with status: %d\n",
                WEXITSTATUS(status));
    }
    return 0;
}

```

Output

```

root@keerthana:~/week4# nano task5.c
root@keerthana:~/week4# gcc task5.c -o task5
root@keerthana:~/week4# ./task5 ls
Parent Process PID: 499
Created Child PID: 500
Child Process PID: 500
task1    task2.c  task4    task5.c    week4_T2
task1.c  task3    task4.c  week4_T1   week4_T2.c
task2    task3.c  task5    week4_T1.c
Child exited with status: 0
root@keerthana:~/week4# |

```

```

root@keerthana:~/week4# ./task5 ls @l
Parent Process PID: 502
Created Child PID: 503
Child Process PID: 503
ls: cannot access '@l': No such file or directory
Child exited with status: 2
root@keerthana:~/week4#

```

Observation

The parent process successfully created a child process, and the child executed the given command. An invalid command argument produced an error and a non-zero exit status.

Result

The command was successfully executed by the child process using `exec()`, and the parent process waited for its completion.